

CASPER@night — Agilent/VnmrJ Quickstart

(SIU Carbondale — 400 MHz DD2, VnmrJ 3.2)

John W. Blanchard (jwbquantum@gmail.com) Reza Ebadi Claude (Anthropic)

Software v0.6.0 — September 15, 2026

1 What this is

This guide covers one specific session: a receive-only spin-noise measurement on the SIU Carbondale 400 MHz Agilent DD2 (VnmrJ 3.2, 5 mm HCN probe, room temperature). It is the Agilent leg of the CASPER@night network — the same physics protocol the Bruker fleet runs, recording the Guéron absorption dip of water on a room-temperature probe — acquired by hand with ordinary VnmrJ tools (`s2pul`, `go`, `svf`) and converted into a network bundle afterward on any laptop. Nothing in the session pulses the sample above tune-and-match power: the calibration steps use $\sim 1^\circ$ tips at your ordinary observe power, and the noise blocks use `pw=0` at the bottom of the transmitter power scale. One thing to know going in: **this path has run on real hardware on one instrument — this one**. Two sessions on 2026-09-14 (68 and 66 experiments) were packed, validated and uploaded with zero reader warnings, so the converter has parsed real files from one instrument of this console family and no other. If your files make it choke, that is still data we need — ugly output is useful output, and Section 4 says what to send. You never need the GitHub repository, git, or a README: this PDF plus the `config.json` attached to the same e-mail is everything.

What you need

1. **Any laptop or desktop with Python 3.6 or newer** — macOS, Linux, and Windows all work; nothing is installed on the spectrometer. Check with:

```
python3 --version
```

2. **The software — one download, no git**. Open

```
https://github.com/CASPER-Night/spin-noise-network/releases/latest
```

and under *Assets* click **Source code (zip)**. Unzip it anywhere.

3. **The `config.json` from John's e-mail** — drop it into the unzipped folder at exactly `uploader/config.json` (next to `upload_bundle.py`). It is your facility's upload key — treat it like a password.
4. **A 5 mm tube of 90% H_2O / 10% D_2O , lightly doped** — a retired shim/lineshape sample is fine, as long as you know (or can best-guess) the composition. The H_2O fraction *is* the measurement.
5. **About 1 hour of instrument time** — roughly 15 min of setup you attend, then ≥ 30 min of noise blocks the console mostly runs itself.

2 At the spectrometer

Everything below is standard VnmrJ operation — no macro, nothing installed on the console. Save every experiment with `svf` into one session directory, one `.fid` per step; the leading number in each save name is what the converter reads (the rest of the name is free).

2.1 Setup (nothing new here, ~15 min)

Insert the water tube, set or record the temperature (298 K if VT is off and the bore is at room temperature), and let it equilibrate a few minutes. Tune and match ^1H by your normal manual wobble procedure — no autotune on this probe, and none needed. Shim as usual, with one caution. On a near-neat water tube the water line may sit at ~ 30 Hz wide and asymmetric, and gradient shimming cannot narrow it — that is radiation damping, the probe's back-action on the spins, not a shim fault. On this instrument, identical shims and identical acquisition gave an 8.5 Hz symmetric line on a 1% $\text{H}_2\text{O}/\text{D}_2\text{O}$ Gd-doped tube and a 32 Hz asymmetric line on the 90/10 tube; three gradient-shim iterations on the 90/10 tube moved z3–z5 by several hundred DAC units each and changed the linewidth by under 0.2 Hz. Judge shim quality on the lock signal or on a dilute/doped tube, not on the near-neat water line, and do not judge a shim set by the size of its DAC values — the set that gave the 8.5 Hz line had z3 and z3y past ± 8000 . The broad line is the regime that produces a large noise feature, so leave it as it is.

Lock: run with the lock **on** during the noise blocks unless you have a reason not to. On this instrument the locked session drifted 0.25 Hz between opening and closing reference over a 38 min bracket, with an unchanged lineshape; the unlocked session drifted 0.56 Hz over a 78 min bracket. That is 0.0065 versus 0.0072 Hz/min — the same drift rate within what two brackets can show — so the two sessions do not by themselves measure what the lock buys. (`alock='n'` in the block parameters below only disables automatic re-locking; it does not release an established lock.) Either way, **record the state** — the questionnaire asks for it. Calibrate the ^1H 90° pulse or take the probe-file value, and **write down pw90 and tpwr** — the questionnaire wants both, and the closing reference must reuse them. Start from your standard ^1H water setup in `s2pul` and leave `sw` at your usual value throughout.

2.2 Gain ladder (~5 min)

Four one-scan 1Ds at ascending receiver gain, with a tiny flip so nothing saturates. First find your ladder maximum: the highest **gain** at which a tiny-flip 1D of this tube does not overflow the receiver. **It is strongly sample-dependent** — on this probe 60 dB was fine on a 1% H_2O Gd-doped tube and a 90/10 $\text{H}_2\text{O}/\text{D}_2\text{O}$ tube needed 40 dB — so the ladder is scaled to your maximum, not fixed at 0/20/40/60: four gains spaced evenly from 0 to the maximum, rounded to whole dB (legal gains are integer dB), e.g. 0/20/40/60 for a 60 dB maximum or 0/14/26/40 for 40 dB. After each **gain=** check the value the console stored — it is what the saved `procpar` carries: asked for 13.3 and 26.7 dB, this console stored 14 and 26. Set **pw** to `pw90/90` — $0.11\ \mu\text{s}$ for a $10\ \mu\text{s}$ `pw90`, about a 1° tip — and **tpwr** to your calibrated observe power (57 below stands in for yours). Write both values down: the two references in 2.3 and 2.5 must use exactly these. Never use **gain='n'** (autogain), and wait for each acquisition to finish before saving. Shown for a 60 dB maximum:

```
pw=0.11
tpwr=57
nt=1
gain=0
go
```

```
svf('10_sn_ladder_a')
gain=20
go
svf('14_sn_ladder_b')
gain=40
go
svf('15_sn_ladder_c')
gain=60
go
svf('16_sn_ladder_d')
```

Read every `gain=60` below as your ladder-maximum value, and note the four values the console stored in the questionnaire.

2.3 Opening reference (~1 min)

One more tiny-flip 1D at the ladder-max gain, with a 2 s record. `pw` and `tpwr` are the two values you wrote down in 2.2 (the same stand-ins below):

```
pw=0.11
tpwr=57
at=2
gain=60
go
svf('11_sn_ref_open')
```

2.4 Noise blocks (≥ 30 min — the point of the session)

No pulse at all — these lines are the receiver-only idiom of Agilent's own `cryo_noisetest` macro, with `tpwr=-16` the bottom of the standard transmitter power scale:

```
pw=0
tpwr=-16
nt=1
ss=0
pad=0
wshim='n'
alock='n'
gain=60
at=30
go
svf('12_sn_noise')
```

Then repeat until you have at least 30 minutes of noise data in total — about 60 blocks at `at=30` — counting up from 17 (13 is reserved for the closing reference):

```
go
svf('17_sn_noise')
go
svf('18_sn_noise')
```

Three notes — two settled on this console, one still open:

- **Record length.** The console caps a record at a point count, $np \leq 524288$, so the longest record is $at_{\max} = 524288/(2\,sw)$ — about 41 s at $sw = 6410$ Hz. `at=30` with about 60 blocks gives 30 minutes; both sessions here used 30 s records (62 and 60 blocks). A narrower `sw` buys a

proportionally longer record — 3205 Hz allows about 82 s — if your usual water window permits it.

- **Transmitter silence.** Whether the DD2's transmitter chain is measurably silent at `pw=0 tpwr=-16` has not been checked — no spectrum analyser was available on 2026-09-14. If the noise floor shows something odd — a carrier spike, a ridge — do not chase it: finish the session and describe it in the questionnaire notes.
- **Save names.** `svf` accepts names starting with a digit on this console — both sessions saved every experiment under the `NN_...` pattern. Only the leading number is parsed.

2.5 Closing reference (~1 min)

Repeat 2.3 exactly — same `at`, same `gain`, and `pw` and `tpwr` back at the two values you wrote down in 2.2 (the noise blocks left `pw=0 tpwr=-16` behind, so both must be restored; do not retype the stand-in numbers below unless they are yours). The two references are a matched drift bracket, and a closing reference at a different flip angle cannot be compared with the opening one — the first session here took them at `pw=0.05` and `0.1125 μs`, a $2.25\times$ mismatch in flip angle, for exactly this reason. Save as 13:

```
pw=0.11
tpwr=57
at=2
gain=60
go
svf('13_sn_ref_close')
```

That is the whole session. An external acquisition driver — a Python 2.6 script that runs on the console workstation itself, submits one block at a time to a running VnmrJ through `listenon/send2Vnmr`, and waits for each `.fid` to land before submitting the next — ran the second session here end to end on 2026-09-14 (66 of 66 blocks, unattended). It is being contributed to `vendors/agilent/`; until it ships there, the manual steps above are the recommended path. The draft VnmrJ macro (`spin_noise_run.mac`) has still not run on hardware.

3 Back at your desk

3.1 Copy the data

Copy the whole session directory — every `NN_*.fid` folder — onto the laptop, any path. A USB stick is fine; the console never needs internet access. Copy the console's `/vnmr/vnmrrev` (three lines of text) into the top of that same directory as `vnmrrev`. The packer takes the VnmrJ version from the `"vnmrj_version"` line of `answers.json` when that line is present — the template below pre-fills SIU's — and reads this copy only when the line is left out, so the copy is the fallback that lets you drop the line rather than retype the version.

3.2 Fill in `answers.json`

In the unzipped `spin-noise-network` folder, create `answers.json` with the content below — it is pre-filled for SIU, so only the `FILL IN` lines (and `"locked"`, `"p90_us"`, and `"vt_setpoint_k"` if yours differed) are yours to edit:

```
{
  "vendor": "agilent",
  "run_mode": "external-acquisition",
```

```

"facility": {
  "institution": "Southern Illinois University",
  "city": "Carbondale, Illinois",
  "country": "USA",
  "facility_slug": "siu-carbondale",
  "contact_email": "bgoodson@chem.siu.edu",
  "contact_consent": true
},
"sample": {
  "description": "FILL IN (e.g. retired lineshape sample, 90% H2O / 10% D2O, Gd-doped)",
  "h2o_fraction_pct": 90,
  "d2o_pct": 10,
  "additives": "FILL IN (dopant, if known)",
  "tube_od_mm": 5,
  "sample_volume_ul": 550,
  "vt_setpoint_k": 298
},
"environment": {
  "locked": true,
  "operator_notes": "FILL IN (date; anything unusual during the run)"
},
"spectrometer": {
  "probe_type": "RT",
  "console": "Agilent DD2 400",
  "probe_string": "5 mm HCN"
},
"instrument": {
  "vnmrj_version": "3.2 Revision A",
  "spectrometer_model": "Agilent DD2 400",
  "field_state_notes": "FILL IN (lock on/off during the noise blocks; z0 stable?)"
},
"calibration": {
  "p90_us": 10.0,
  "p90_power_db_or_w": "FILL IN (tpwr, dB)",
  "topshim_ok": false,
  "rg_ladder": [
    { "expno": 10, "rg": 1.0, "tip_deg": 1.0 },
    { "expno": 14, "rg": 10.0, "tip_deg": 1.0 },
    { "expno": 15, "rg": 100.0, "tip_deg": 1.0 },
    { "expno": 16, "rg": 1000.0, "tip_deg": 1.0 }
  ]
},
"experiments": [
  { "expno": 10, "role": "rg_ladder" },
  { "expno": 14, "role": "rg_ladder" },
  { "expno": 15, "role": "rg_ladder" },
  { "expno": 16, "role": "rg_ladder" },
  { "expno": 11, "role": "reference_open" },
  { "expno": 12, "role": "noise" },
  { "expno": 17, "role": "noise" },
  { "expno": 18, "role": "noise" },
  { "expno": 13, "role": "reference_close" }
]
}

```

Two mechanical notes: add one { "expno": NN, "role": "noise" } line per noise block you saved beyond 18, and the ladder "rg" values are $10^{\text{dB}/20}$ of the four gains *as the console stored them* — pre-filled for 0/20/40/60 dB, so correct them if your ladder used different values (for 0/14/26/40: 1, 5.01, 19.95, 100).

3.3 Pack, check, upload

Three commands, run from inside the unzipped **spin-noise-network** folder:

```
python3 packer/pack_bundle.py --vendor agilent --answers answers.json
/path/to/the/data/folder
```

Success looks like: the packer validates the session and prints the path of one **spinnoise_siu-carbondale_*.zip** — and if anything is missing, it lists exactly what, in plain text, so you fix it and re-run.

```
python3 uploader/upload_bundle.py --selftest spinnoise_*.zip
```

Success looks like: the last line reads **RESULT: PASS** — entirely local, no network involved yet.

```
python3 uploader/upload_bundle.py spinnoise_*.zip
```

Success looks like: the last line reads **RECEIPT: <id>** — the server has your bundle, checksum-verified on arrival; uploads are chunked and resumable, so if the connection drops, re-run the same command and it picks up where it left off.

4 If anything goes wrong

One rule: send it, don't fix it

Send the **exact error text** (copy-paste, not a paraphrase) and the file it choked on — the offending **NN_*.fid** folder zipped, or the bundle zip — to jwbquantum@gmail.com. The converter has met one real Agilent instrument — this one, two sessions — and every further session is how it becomes trustworthy beyond that. Nothing you can do here damages anything — the software only reads your data, the uploader never deletes your zip, and the worst case at every step is re-running one command. If the whole chain fights you, zip the entire session directory and e-mail that (a share link is fine if it is large); we take it from there.

Contact: John W. Blanchard (jwbquantum@gmail.com) — CASPER@night software v0.6.0 — September 15, 2026. Your raw data remain yours, and contributing facilities are co-authors on network publications their data enter.